ANALISADOR LÉXICO PARA A LINGUAGEM MINI_PASCAL

João Paulo Sandes Brito
Universidade Estadual do Sudoeste da Bahia – UESB

Resumo

O trabalho apresenta um analisador léxico para Mini_Pascal. O programa foi escrito em Java 17, possui interface em Swing e gera pares no formato lexema–token. Foram usados seis arquivos de teste, incluindo um conjunto com erros léxicos.

Palavras-chave: análise léxica; Java; Mini_Pascal.

LEXICAL ANALYZER FOR THE MINI_PASCAL LANGUAGE

Abstract

This paper presents a lexical analyzer for Mini_Pascal. The program was written in Java 17, provides a Swing interface and generates lexeme–token pairs. Six input files were used for testing, including lexical errors.

Keywords: lexical analysis; Java; Mini_Pascal.

1 OBJETIVO

O analisador lê um arquivo-fonte e separa seu conteúdo em tokens. Espaços, quebras de linha e comentários são ignorados. O resultado pode ser visto na interface ou salvo em arquivo texto.

A implementação usa transições determinísticas e mantém a linha e a coluna de cada lexema. Essa organização corresponde ao funcionamento usual da etapa léxica de um compilador [1].

2 CLASSES LÉXICAS

Tabela 1: Classes reconhecidas

Classe	Exemplos	Token
Palavra reservada	`program`, `begin`, `integer`	Palavra reservada
Identificador	`x`, `var2`, `media_geral`	Identificador
Número	`13`, `1.33`, `24.40e-04`	Inteiro ou real
Constante	`'A'`, `"texto"`	Caractere ou string
Operador	`+`, `mod`, `>=`, `and`	Aritmético, relacional ou lógico
Símbolo	`( ) , ; :`, `:=`, `.`	Especial, atribuição ou fim

Palavras reservadas não diferenciam maiúsculas de minúsculas, mas o lexema original é mantido. Os termos `read`, `write` e `writeln` seguem o exemplo da atividade e são classificados como identificadores. O símbolo = é tratado como operador relacional.

3 IMPLEMENTAÇÃO

Tabela 2: Principais classes do programa

Classe	Função
`Lexer`	Reconhecer lexemas e produzir tokens.
`Token/TokenType`	Guardar lexema, tipo, linha, coluna e mensagem.
`AnalysisFileService`	Ler a entrada e gravar a saída.
`MainFrame`	Exibir a interface gráfica.
`Main`	Iniciar o programa.

O método `nextToken()` lê um token por chamada. O método `scanAll()` repete a operação até o fim do arquivo. A decisão inicial depende do primeiro caractere do lexema:

```
letra ou '_' -> identificador ou palavra reservada
digito       -> numero inteiro ou real
aspas        -> constante
operador     -> forma simples ou composta
outro        -> erro lexico
```

4 ERROS LÉXICOS

Os erros registram lexema, linha, coluna e mensagem. Depois do registro, a leitura continua no próximo ponto válido. São detectados símbolos desconhecidos, expoentes incompletos, números unidos a identificadores, comentários ou strings sem fechamento e constantes de caractere inválidas.

5 TESTES

Tabela 3: Casos executados

Caso	Conteúdo	Tokens	Erros
Teste 01	Programa básico	50	0
Teste 02	Números e constantes	48	0
Teste 03	Operadores	74	0
Teste 04	Comentários	15	0
Teste 05	Entradas inválidas	30	5
Teste 06	Maiúsculas, sublinhado e Unicode	32	0

Os seis arquivos produziram 249 tokens. As saídas foram comparadas com os arquivos esperados. O teste automatizado também executou vinte verificações.

6 EXECUÇÃO

No Windows, o programa pode ser aberto por `executar.bat`. O JAR também aceita entrada e saída pela linha de comando:

```
java -jar dist\MiniPascal-Lexico.jar entrada.txt saida.txt
```

É necessário ter Java 17 ou superior. Não é preciso usar uma IDE.

7 CONSIDERAÇÕES FINAIS

O programa reconhece as classes previstas, informa erros sem interromper toda a leitura e gera a listagem lexema–token. Os arquivos de teste e o JAR acompanham o código-fonte.

REFERÊNCIAS

[1] AHO, Alfred V.; LAM, Monica S.; SETHI, Ravi; ULLMAN, Jeffrey D. **Compilers: principles, techniques, and tools**. 2. ed. Boston: Pearson Addison-Wesley, 2007.